

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



Các công nghệ lập trình hiện đại

**Tìm hiểu LlamaIndex và ứng dụng RAG
trong hệ thống hỏi đáp tài liệu đào tạo**

GVHD: Phạm Thị Vương

SV: Huỳnh Thanh Tiến 3123410374
Phan Hữu Kha 3123560038
Võ Thị Như Ý 3123410439
Trần Hà Thanh Thùy 3123560087

TP. HỒ CHÍ MINH, THÁNG 10/2026

MỤC LỤC

1	Giới thiệu đề tài	1
1.1	Bối cảnh và động cơ	1
1.2	Mục tiêu của đề tài	1
1.3	Phạm vi báo cáo	2
1.4	Phương pháp thực hiện	2
1.5	Bố cục báo cáo	2
2	Cơ sở lý thuyết	4
2.1	Mô hình ngôn ngữ lớn và những hạn chế khi hỏi đáp tài liệu riêng . . .	4
2.2	Retrieval-Augmented Generation (RAG)	4
2.2.1	Ý tưởng	4
2.2.2	Hai pha của một hệ RAG	5
2.3	Embedding và độ tương đồng vector	5
2.3.1	Vector embedding	5
2.3.2	Độ đo cosine	5
2.3.3	Mô hình embedding tiếng Việt được chọn	6
2.4	Cắt đoạn văn bản (chunking)	6
2.4.1	Vì sao phải cắt	6
2.4.2	Các chiến lược cắt	6
2.5	Chỉ mục vector và truy hồi top- k	7
2.6	Tổng hợp câu trả lời (Response Synthesis)	7
2.7	Xếp hạng lại (reranking)	8
2.8	Các khái niệm cốt lõi của LlamaIndex	8
2.9	Nhận dạng ký tự quang học (OCR)	9
2.10	Mô hình chạy cục bộ: Ollama và Qwen3	10
2.11	Đánh giá chất lượng truy hồi	10
3	Thiết lập môi trường và kiến trúc dự án	12
3.1	Công cụ cần thiết	12
3.1.1	Thư viện Python	12
3.1.2	Phụ thuộc hệ thống	12
3.2	Cài đặt và Hello World	13
3.2.1	Các bước cài đặt	13
3.2.2	Hello World	13
3.3	Cấu trúc thư mục dự án	14
3.4	Cấu hình tập trung và hai profile	15

3.5	Phần cứng và bộ nhớ GPU	16
3.6	Các script hỗ trợ	17
3.7	Minh chứng kiến trúc	17
4	Bản chất công nghệ — Bắt buộc lỗi	19
4.1	Nạp dữ liệu (Ingestion)	19
4.1.1	Cơ chế	19
4.1.2	Mã nguồn	19
4.1.3	Kết quả chạy thật	20
4.1.4	Nhận xét chất lượng OCR	21
4.1.5	Vị trí trong đồ án	21
4.2	Cắt đoạn theo Điều (Node)	21
4.2.1	Cơ chế	21
4.2.2	Mã nguồn	22
4.2.3	Kết quả chạy thật	23
4.2.4	Phân tích kết quả	23
4.2.5	Hạn chế đã biết	24
4.2.6	Vị trí trong đồ án	24
4.3	Lập chỉ mục (Index)	24
4.3.1	Cơ chế	24
4.3.2	Mã nguồn	25
4.3.3	Kết quả chạy thật	26
4.3.4	Vị trí trong đồ án	26
4.4	Truy hồi (Retriever) và xếp hạng lại	27
4.4.1	Cơ chế	27
4.4.2	Mã nguồn	27
4.4.3	Kết quả chạy thật	28
4.5	Tổng hợp câu trả lời (Query Engine)	28
4.5.1	Cơ chế	28
4.5.2	Mã nguồn	29
4.5.3	Kết quả chạy thật	30
4.6	Tổng kết	30
5	Đánh giá	32
5.1	Đánh giá truy hồi	32
5.1.1	Phương pháp	32
5.1.2	Kết quả	33
5.1.3	Phân tích	33

5.2	Đánh giá câu trả lời	34
5.2.1	Dữ liệu	34
5.2.2	Các trường hợp tiêu biểu	35
5.2.3	Phân tích	36
5.3	Hạn chế	36
5.4	Hướng phát triển	37
5.5	Kết luận	38

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và động cơ

Quy chế đào tạo là văn bản mà sinh viên tra cứu thường xuyên trong suốt quá trình học: thời gian tối đa để hoàn thành khóa học, điều kiện dự thi, cách tính điểm học phần, điều kiện xét tốt nghiệp, thủ tục nghỉ học tạm thời hay chuyển ngành. Tại Trường Đại học Sài Gòn (SGU), các quy định này nằm trong những văn bản pháp quy dài, được ban hành dưới dạng file PDF. Một câu hỏi rất ngắn của sinh viên (ví dụ: “nghỉ học bao nhiêu buổi thì bị cấm thi?”) thường buộc người hỏi phải đọc nhiều Điều khác nhau mới tìm được câu trả lời chính xác.

Hai cách xử lý thông thường đều có hạn chế. Tìm kiếm theo từ khóa trong file PDF chỉ khớp khi người dùng dùng đúng thuật ngữ của văn bản; nếu người dùng diễn đạt khác (“cấm thi” thay cho “không đủ điều kiện dự thi”) thì không tìm thấy. Hỏi trực tiếp một mô hình ngôn ngữ lớn (LLM) thì câu trả lời nghe rất trôi chảy nhưng không có căn cứ vào quy chế của SGU, và mô hình có thể bịa ra số liệu hoặc điều khoản không tồn tại.

Kỹ thuật **RAG** (Retrieval-Augmented Generation — sinh văn bản có tăng cường truy hồi) giải quyết đúng khoảng trống này: trước khi trả lời, hệ thống tìm những đoạn quy chế liên quan nhất, rồi yêu cầu LLM chỉ trả lời dựa trên các đoạn đó. **LlamaIndex** là một framework mã nguồn mở cung cấp sẵn các thành phần cho toàn bộ chuỗi xử lý này (đọc dữ liệu, cắt đoạn, lập chỉ mục, truy hồi, tổng hợp câu trả lời), nên được chọn làm nền tảng của đồ án.

1.2 Mục tiêu của đề tài

Mục tiêu chung của đồ án là tìm hiểu LlamaIndex và vận dụng kỹ thuật RAG để xây dựng hệ thống hỏi đáp trên tài liệu đào tạo của SGU, chạy hoàn toàn với các mô hình cục bộ (không gửi dữ liệu ra dịch vụ bên ngoài). Cụ thể:

- (a) Hiểu được vai trò và cách hoạt động của từng thành phần lõi trong LlamaIndex: Document, Node, Index, Retriever, QueryEngine/Response Synthesis.
- (b) Xây dựng bước nạp dữ liệu từ PDF bản scan tiếng Việt (OCR) thành các Document có metadata.

- (c) Xây dựng bước cắt văn bản theo cấu trúc pháp lý của quy chế (từng *Điều*) thành các Node kèm số Điều.
- (d) Lập chỉ mục vector bằng mô hình embedding tiếng Việt, truy hồi top- k và sinh câu trả lời có trích dẫn số Điều bằng LLM chạy qua Ollama.
- (e) Đo chất lượng truy hồi bằng bộ câu hỏi có nhãn ($\text{Recall}@k$, MRR) và phân tích các lỗi của câu trả lời.

1.3 Phạm vi báo cáo

Báo cáo bao gồm phần *Bắt buộc lỗi* của đề án: kiến thức nền, thiết lập môi trường, kiến trúc, mã nguồn và kết quả chạy thật của cả sáu giai đoạn pipeline (nạp dữ liệu, cắt Node, lập chỉ mục, truy hồi, tổng hợp câu trả lời, đánh giá). Ở thời điểm viết, toàn bộ sáu giai đoạn đã có mã nguồn chạy được trên một văn bản là *Quy chế đào tạo trình độ đại học của SGU năm 2021*; hệ thống hỏi đáp chạy được qua dòng lệnh (`src/pipeline.py`).

Reranker là phần *chọn thêm*: mã nguồn đã có (bật ở profile máy chủ) và báo cáo có đo tác động của nó lên truy hồi, nhưng không đi sâu. Những phần chưa làm — giao diện web Gradio, mở rộng sang nhiều văn bản (Quyết định sửa đổi 3183/2025, các văn bản công tác sinh viên) và bộ định tuyến (router) giữa các nguồn — được đánh dấu rõ là chưa triển khai.

1.4 Phương pháp thực hiện

Đề án làm theo từng giai đoạn nhỏ; mỗi giai đoạn phải chạy được và cho kết quả quan sát được trước khi sang giai đoạn kế tiếp. Số liệu trong Chương 3, 4 và 5 đều lấy từ việc chạy thật các script trong repository, không dùng số liệu minh họa. Khi một module chưa tồn tại, báo cáo nói rõ là chưa triển khai thay vì mô tả như thể đã xong.

1.5 Bố cục báo cáo

- **Chương 1** — Giới thiệu đề tài (chương này).
- **Chương 2** — Cơ sở lý thuyết: LLM và hạn chế, RAG, embedding, cắt đoạn, chỉ mục vector, tổng hợp câu trả lời, các khái niệm của LlamaIndex, OCR, và độ đo

đánh giá truy hồi.

- **Chương 3** — Thiết lập môi trường và kiến trúc dự án.
- **Chương 4** — Bản chất công nghệ: mã nguồn thật và kết quả chạy thật của sáu module lỗi.
- **Chương 5** — Đánh giá: số liệu truy hồi trên hai bộ câu hỏi, so sánh hai profile và tác động của reranker, phân tích lỗi câu trả lời, hạn chế và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Mô hình ngôn ngữ lớn và những hạn chế khi hỏi đáp tài liệu riêng

Mô hình ngôn ngữ lớn (Large Language Model, LLM) là mạng nơ-ron được huấn luyện trên lượng văn bản rất lớn để dự đoán token tiếp theo. Nhờ đó LLM có khả năng hiểu câu hỏi, diễn đạt lại, tóm tắt và suy luận đơn giản. Tuy nhiên, khi dùng LLM để trả lời về một văn bản nội bộ như quy chế của SGU, có bốn hạn chế cần xử lý:

1. **Thiếu tri thức riêng.** Quy chế của một trường cụ thể, nhất là bản ban hành gần đây, hầu như không nằm trong dữ liệu huấn luyện, hoặc chỉ xuất hiện rất mờ nhạt.
2. **Tri thức bị đóng băng.** Kiến thức của mô hình dừng ở thời điểm huấn luyện; khi quy chế được sửa đổi, mô hình không tự cập nhật.
3. **Ảo giác (hallucination).** Mô hình có xu hướng sinh ra câu trả lời nghe hợp lý nhưng sai, ví dụ bịa số tín chỉ hoặc số Điều.
4. **Khó kiểm chứng.** Câu trả lời không kèm nguồn nên người dùng không biết đối chiếu ở đâu.

Có thể khắc phục bằng cách tinh chỉnh (fine-tuning) mô hình trên văn bản quy chế, nhưng cách này tốn kém, phải huấn luyện lại mỗi khi văn bản đổi, và vẫn không loại bỏ hoàn toàn ảo giác. RAG đi theo hướng khác: giữ nguyên mô hình, đưa tri thức cần thiết vào *ngữ cảnh* (prompt) tại thời điểm hỏi.

2.2 Retrieval-Augmented Generation (RAG)

2.2.1 Ý tưởng

RAG tách bài toán thành hai bước: (1) *truy hỏi* — từ kho tài liệu, tìm ra một số ít đoạn liên quan nhất tới câu hỏi; (2) *sinh* — đưa câu hỏi cùng các đoạn đó cho LLM và yêu cầu trả lời dựa trên chúng. Nếu ký hiệu q là câu hỏi, $\mathcal{D}$ là tập các đoạn văn bản trong kho, thì:

$$C = \text{Retrieve}(q, \mathcal{D}, k), \quad a = \text{LLM}(\text{prompt}(q, C)), \quad (2.1)$$

trong đó $C \subset \mathcal{D}$ là k đoạn được chọn và a là câu trả lời. Vì C lấy trực tiếp từ quy chế nên câu trả lời có căn cứ, và hệ thống có thể trích dẫn nguồn (số Điều, số trang) cho người dùng kiểm tra.

2.2.2 Hai pha của một hệ RAG

Một hệ RAG hoạt động theo hai pha có tần suất chạy khác nhau:

- **Pha chuẩn bị (offline), chạy khi tài liệu thay đổi:** nạp tài liệu $\rightarrow$ cắt thành các đoạn $\rightarrow$ biến mỗi đoạn thành vector embedding $\rightarrow$ lưu vào chỉ mục.
- **Pha truy vấn (online), chạy mỗi câu hỏi:** embedding câu hỏi $\rightarrow$ tìm k đoạn gần nhất $\rightarrow$ ghép prompt $\rightarrow$ LLM sinh câu trả lời.

Chất lượng của hệ thống bị giới hạn bởi *đoạn văn tìm được*: nếu bước truy hồi bỏ sót Điều đúng thì LLM dù tốt đến đâu cũng không thể trả lời đúng. Vì vậy phần lớn công sức của đồ án tập trung vào các bước chuẩn bị dữ liệu (nạp, cắt đoạn) và truy hồi.

2.3 Embedding và độ tương đồng vector

2.3.1 Vector embedding

Embedding là phép ánh xạ một đoạn văn bản thành một vector số thực $\mathbf{v} \in \mathbb{R}^d$ sao cho các đoạn có nghĩa gần nhau nằm gần nhau trong không gian vector. Với mô hình dạng bi-encoder, câu hỏi và đoạn văn được mã hóa *độc lập* bởi cùng một bộ mã hóa; nhờ vậy vector của toàn bộ kho tài liệu có thể tính trước và lưu lại, khi có câu hỏi chỉ cần mã hóa riêng câu hỏi.

2.3.2 Độ đo cosine

Độ tương đồng giữa vector câu hỏi $\mathbf{q}$ và vector đoạn văn $\mathbf{d}$ thường được đo bằng cosine:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^d q_i d_i}{\sqrt{\sum_{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d d_i^2}}. \quad (2.2)$$

Giá trị càng gần 1 thì hai đoạn càng gần nhau về ngữ nghĩa. Khác với tìm theo từ khóa, so khớp vector cho phép “cắm thi” tìm được đoạn viết “không đủ điều kiện dự thi” vì hai cụm có embedding gần nhau.

2.3.3 Mô hình embedding tiếng Việt được chọn

Chất lượng truy hồi phụ thuộc mạnh vào việc mô hình embedding có được huấn luyện cho tiếng Việt hay không. Theo cấu hình trong `src/config.py`, đề án dùng:

- Profile chạy cục bộ: `bkai-foundation-models/vietnamese-bi-encoder`, vector 768 chiều;
- Profile máy chủ: `AITeamVN/Vietnamese_Embedding`, vector 1024 chiều.

Cả hai đều được nạp qua `llama-index-embeddings-huggingface` (lớp `HuggingFaceEmbedding`, mặc định chuẩn hóa vector về độ dài 1 nên tích vô hướng chính là cosine). Vì hai mô hình cho vector khác số chiều, chỉ mục đã lập bằng mô hình này không dùng được với mô hình kia; đổi mô hình embedding đồng nghĩa phải lập lại chỉ mục. Chương 5 so sánh chất lượng truy hồi của hai mô hình trên cùng bộ câu hỏi.

2.4 Cắt đoạn văn bản (chunking)

2.4.1 Vì sao phải cắt

Có hai lý do phải cắt tài liệu thành các đoạn nhỏ. Thứ nhất, cửa sổ ngữ cảnh của LLM có hạn, và càng nhét nhiều văn bản không liên quan thì câu trả lời càng dễ lệch. Thứ hai, một vector embedding cho cả tài liệu sẽ “pha loãng” mọi ý; mỗi đoạn nên xoay quanh một chủ đề để vector của nó đại diện đúng.

2.4.2 Các chiến lược cắt

Chiến lược	Ưu điểm	Nhược điểm
Cắt theo độ dài cố định	Đơn giản, kích thước đều	Có thể cắt ngang câu hoặc ngang điều khoản
Cắt theo câu (sentence splitter)	Không gãy câu, có chồng lấp (overlap)	Vẫn không biết ranh giới điều khoản
Cắt theo cấu trúc văn bản	Mỗi đoạn là một đơn vị nghĩa trọn vẹn	Phụ thuộc định dạng văn bản, cần quy tắc riêng

Bảng 2.1: So sánh ba chiến lược cắt đoạn.

Quy chế đào tạo có cấu trúc rất rõ: văn bản chia thành các *Điều*, mỗi *Điều* chia thành khoản và điểm. Vì vậy đồ án chọn **cắt theo Điều** làm chiến lược chính; riêng những *Điều* dài vượt ngưỡng thì cắt tiếp theo câu. Cách này giữ trọn ngữ cảnh pháp lý và cho phép gắn số *Điều* vào metadata để trích dẫn nguồn.

2.5 Chỉ mục vector và truy hồi top- k

Sau khi có vector cho mọi đoạn, hệ thống cần lưu chúng theo cách tìm kiếm nhanh. *Chỉ mục vector* (vector index) lưu các cặp (vector, đoạn văn, metadata). Khi có câu hỏi, bộ truy hồi (retriever) tính độ tương đồng giữa vector câu hỏi và các vector trong chỉ mục rồi trả về k đoạn có điểm cao nhất (top- k). Với kho nhỏ như một quy chế (vài chục đến vài trăm đoạn), việc so sánh vét cạn với mọi vector là đủ nhanh; các cấu trúc tìm kiếm xấp xỉ chỉ cần khi kho rất lớn.

Tham số k là sự đánh đổi: k nhỏ để bỏ sót đoạn cần thiết, k lớn đưa nhiều nhiễu vào prompt. Cấu hình hiện có trong `src/config.py` đặt `similarity_top_k = 10`.

2.6 Tổng hợp câu trả lời (Response Synthesis)

Sau truy hồi, các đoạn tìm được phải được đưa cho LLM để sinh câu trả lời. LlamaIndex gọi bước này là *response synthesis* và cung cấp nhiều chế độ (`response_mode`). Các chế độ thường gặp:

- `refine`: gọi LLM lần lượt từng đoạn, mỗi lần tinh chỉnh câu trả lời cũ bằng đoạn mới; chính xác nhưng tốn nhiều lượt gọi.
- `compact`: gộp nhiều đoạn vào cùng một prompt sao cho vừa cửa sổ ngữ cảnh rồi mới gọi LLM, giảm số lượt gọi so với `refine`.
- `tree_summarize`: tóm tắt theo cấu trúc cây, phù hợp câu hỏi tổng hợp trên nhiều đoạn.
- `simple_summarize`: cắt bớt để nhét mọi đoạn vào một lần gọi.

Đồ án dùng `response_mode = "compact"` và `streaming = True`: với $k = 10$ *Điều* (hoặc 3 *Điều* sau rerank), mỗi *Điều* trung bình khoảng 1 500 ký tự,

toàn bộ ngữ cảnh vừa một lần gọi LLM, nên `compact` thường chỉ gọi LLM một lần; `streaming` cho phép in từng token ra màn hình ngay khi mô hình sinh.

2.7 Xếp hạng lại (reranking)

Mô hình embedding dạng bi-encoder mã hóa câu hỏi và đoạn văn *riêng rẽ*, nên nhanh nhưng chỉ so khớp ở mức “ý chính”. Mô hình *cross-encoder* nhận *cặp* (câu hỏi, đoạn văn) làm một đầu vào duy nhất và chấm một điểm liên quan; nhờ xem cả hai cùng lúc, nó thường đánh giá chính xác hơn, nhưng phải chạy lại cho từng cặp nên chậm hơn nhiều và không thể tính trước. Vì vậy cách dùng phổ biến là hai tầng: bi-encoder lấy nhanh top- k ứng viên, rồi cross-encoder chấm lại và chỉ giữ top- n ($n < k$) đưa vào prompt.

Trong đồ án, reranker AITeamVN/Vietnamese_Reranker chỉ bật ở profile máy chủ, với $k = 10$ và $n = 3$. Đây là phần chọn thêm; Chương 5 đo xem nó có cải thiện truy hồi trên dữ liệu của đồ án hay không.

2.8 Các khái niệm cốt lõi của LlamaIndex

LlamaIndex mô hình hóa chuỗi RAG bằng một số đối tượng có tên rõ ràng. Bảng dưới đây tóm tắt và chỉ ra chúng nằm ở đâu trong đồ án.

Khái niệm	Vai trò	Trong đồ án
Document	Một nguồn dữ liệu thô kèm meta-data	<code>ingestion.py</code> (mỗi trang PDF một Document)
Node / TextNode	Đơn vị nhỏ hơn sinh từ Document, là đơn vị được lập chỉ mục	<code>node_parser.py</code> (mỗi Điều một Node)
Index	Cấu trúc lưu Node để truy hồi (ví dụ <code>VectorStoreIndex</code>)	<code>index_builder.py</code> (lưu tại <code>storage/vector_index</code>)
Retriever	Lấy các Node liên quan tới câu hỏi	<code>retriever.py</code> (top- k theo cosine)
Node postprocessor	Xử lý danh sách Node sau truy hồi (lọc, xếp hạng lại)	<code>retriever.py</code> (reranker, profile máy chủ)
Response synthesizer	Ghép Node vào prompt và gọi LLM	<code>query_engine.py</code> (chế độ compact)
QueryEngine	Gộp retriever, postprocessor và synthesizer thành một lệnh hỏi–đáp	<code>query_engine.py</code> (<code>RetrieverQueryEngine</code>)

Bảng 2.2: Các đối tượng chính của LlamaIndex và vị trí trong đồ án.

Điểm mạnh của thiết kế này là mỗi khâu độc lập: có thể đổi mô hình embedding, đổi cách cắt đoạn hoặc đổi chế độ tổng hợp mà không phải viết lại toàn bộ hệ thống.

2.9 Nhận dạng ký tự quang học (OCR)

Bản PDF của Quy chế đào tạo SGU là bản scan: mỗi trang là một ảnh, không có lớp văn bản (text layer). Khi đọc bằng bộ đọc PDF thông thường, hệ thống gần như không lấy được chữ nào. Để có văn bản, cần OCR.

Tesseract là công cụ OCR mã nguồn mở, có gói ngôn ngữ tiếng Việt (`vie`); trong Python được gọi qua thư viện `pytesseract`. Vì Tesseract nhận ảnh làm đầu vào, mỗi trang PDF trước hết được render thành ảnh bằng `pdf2image` (dựa trên `poppler`). Độ phân giải càng cao thì nhận dạng càng tốt nhưng chậm hơn; đồ án dùng 300 dpi.

OCR tiếng Việt luôn có sai sót, nhất là với dấu thanh và chữ in mờ (ví dụ nhầm “NHÂN” thành “NHÀN”). Điều này ảnh hưởng đến các bước sau, đặc biệt khi quy tắc cắt đoạn dựa vào việc nhận đúng từ “Điều”. Phần 4.1 sẽ nêu các lỗi cụ thể quan sát được.

2.10 Mô hình chạy cục bộ: Ollama và Qwen3

Để dữ liệu không đi qua dịch vụ AI bên thứ ba, LLM được chạy qua **Ollama**, một trình phục vụ mô hình mở với API HTTP (mặc định tại `http://localhost:11434`); ở profile máy chủ, Ollama chạy trên một máy GPU thuê riêng và địa chỉ được đọc từ biến `OLLAMA_BASE_URL`. Đồ án dùng họ mô hình Qwen3: `qwen3:4b` cho profile chạy cục bộ và `qwen3:8b` cho profile máy chủ. LlamaIndex kết nối tới Ollama qua gói `llama-index-llms-ollama`. Thời gian chờ mặc định trong cấu hình là 180 giây do mô hình chạy trên phần cứng cá nhân có thể trả lời chậm. Qwen3 có chế độ “suy nghĩ” (sinh chuỗi lập luận trước câu trả lời); đồ án tắt chế độ này (`thinking=False`) để câu trả lời ngắn và nhanh.

2.11 Đánh giá chất lượng truy hỏi

Do chất lượng RAG phụ thuộc đầu tiên vào việc tìm đúng đoạn, đồ án đánh giá riêng bước truy hỏi trước khi đánh giá câu trả lời. Có hai bộ câu hỏi kiểm thử, mỗi câu có hai trường `question` và `dieu_dung` (số Điều chứa đáp án):

- `eval/test_questions_01.json`: 31 câu về Quy chế đào tạo 2021;
- `eval/test_questions_02.json`: 10 câu về các nội dung được *sửa đổi* bởi Quyết định 3183/2025 (rút học phần, số học kỳ, thang điểm chữ B+, C+, D+...).

Nhờ có nhãn, có thể dùng hai độ đo phổ biến. Gọi $n_{i,j}$ là Node đứng thứ j trong danh sách trả về cho câu hỏi thứ i , N là số câu hỏi:

- **Recall@ k** (còn gọi là `hit rate@ $k$`): tỷ lệ câu hỏi mà trong k Node đầu tiên có ít nhất một Node thuộc Điều đúng. Vì mỗi câu chỉ có một Điều đúng, hai cách gọi này trùng nhau:

$$\text{Recall}@k = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\exists j \leq k : \text{dieu}(n_{i,j}) = \text{dieu_dung}_i]. \quad (2.3)$$

- **MRR** (Mean Reciprocal Rank): trung bình nghịch đảo thứ hạng của Node đúng đầu tiên (bằng 0 nếu không tìm thấy):

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}. \quad (2.4)$$

Recall@ k cho biết Điều đúng *có lọt vào* ngữ cảnh đưa cho LLM hay không; MRR cho biết nó *đúng cao* tới đâu. Hai độ đo này chỉ chấm bước truy hồi; chất lượng câu trả lời của LLM được phân tích riêng bằng cách đối chiếu thủ công với văn bản quy chế (Chương 5).

CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN

3.1 Công cụ cần thiết

3.1.1 Thư viện Python

File `requirements.txt` liệt kê tên gói mà không ghi phiên bản. Bảng dưới ghi phiên bản thực tế cài trong `venv` của repository tại thời điểm đo.

Gói	Phiên bản đo được	Mục đích
<code>llama-index-core</code>	0.14.24	Framework RAG
<code>llama-index-embeddings-huggingface</code>	0.8.0	Nạp mô hình embedding
<code>llama-index-llms-ollama</code>	0.11.0	Kết nối LLM qua Ollama
<code>sentence-transformers</code>	6.0.1	Chạy embedding/reranker
<code>torch</code> (cài kèm)	2.14.0 (CUDA 13.0)	Chạy mô hình trên GPU
<code>sentencepiece, protobuf</code>	0.2.2, không đo	Phụ thuộc của tokenizer
<code>pytesseract</code>	0.3.13	Gọi Tesseract từ Python
<code>pdf2image</code>	1.17.0	Render PDF thành ảnh
<code>gradio</code>	6.27.0	Giao diện web (dự kiến)
<code>python-dotenv</code>	1.2.3	Đọc biến môi trường từ <code>.env</code>

Bảng 3.1: Thư viện Python trong `requirements.txt`.

3.1.2 Phụ thuộc hệ thống

- **Tesseract OCR 5.5.3**, đã xác nhận có gói ngôn ngữ `vie`.

- **poppler-utils** (pdftoppm 26.01.0), cần cho `pdf2image`.
- **Ollama** để chạy LLM (script `pull_models.sh` kiểm tra và báo lỗi nếu chưa cài).
- Python 3.14.6 trong `venv`.

3.2 Cài đặt và Hello World

3.2.1 Các bước cài đặt

Trình tự đã dùng trong dự án:

1. Tạo và kích hoạt môi trường ảo Python (`venv`).
2. Cài các gói Python: `pip install -r requirements.txt`.
3. Cài Tesseract (kèm gói ngôn ngữ `vie`) và `poppler-utils` bằng trình quản lý gói của hệ điều hành.
4. Cài Ollama, chạy `ollama serve`, rồi tải mô hình bằng `scripts/pull_models.sh [local|server]`.

3.2.2 Hello World

Script `scripts/hello_world.py` kiểm tra nhanh rằng `LlamaIndex` cài đúng: tạo một `Document`, dựng `SummaryIndex` và đếm số `Document` trong `docstore`. Script này không cần LLM hay mô hình embedding nên chạy được ngay sau khi cài thư viện.

Đoạn mã 3.1: Tạo `Document` và `SummaryIndex` với `LlamaIndex` (`scripts/hello_world.py`).

```
1 from llama_index.core import Document, SummaryIndex
2
3 doc = Document(text = "Điều 15. Sinh viên nghỉ học quá 20% số tiết
    sẽ không đủ điều kiện dự thi.")
4 print("Document tạo thành công!")
5 print("Nội dung:", doc.text)
6 print("ID tự sinh:", doc.doc_id)
7
8 index = SummaryIndex.from_documents([doc])
```

```
9 print("\nIndex build thành công!")
10 print("Số Document trong Index:", len(index.docstore.docs))
```

Kết quả chạy thật:

Đoạn mã 3.2: Kết quả chạy hello_world.py.

```
1 Document tạo thành công!
2 Nội dung: Điều 15. Sinh viên nghỉ học quá 20% số tiết sẽ không đủ
  điều kiện dự thi.
3 ID tự sinh: 3dc12976-3b4e-4711-8798-c7d0237d98a9
4
5 Index build thành công!
6 Số Document trong Index: 1
```

Ý nghĩa: doc_id được LlamaIndex tự sinh dạng UUID, và SummaryIndex lưu Document vào docstore. Câu ví dụ “Điều 15” trong script chỉ là văn bản minh họa tự viết, không phải nội dung trích từ quy chế.

3.3 Cấu trúc thư mục dự án

Đoạn mã 3.3: Cấu trúc thư mục dự án.

```
1 .
2 |-- app/          # Giao diện Gradio (chưa có mã, mới có app/assets)
3 |-- data/
4 |   |-- raw/      # PDF gốc: Quy chế đào tạo 2021, QĐ 3183/2025
5 |   |-- ocr/       # Bản OCR dạng Markdown của Quy chế 2021
6 |   |-- processed/ # (trong)
7 |-- eval/
8 |   |-- evaluate.py      # Do Recall@k
9 |   |-- test_questions_01.json # 31 câu hỏi
10 |  |-- test_questions_02.json # 10 câu hỏi về QĐ sửa đổi 3183/2025
11 |  |-- results/
12 |-- notebooks/ # Chạy thử pipeline, chạy trên GPU thuê
13 |-- scripts/   # hello_world.py, test_ocr.py, check_model_memory.py,
14 |               # pull_models.sh, manager.py
15 |-- src/
16 |   |-- config.py  # RAGConfig + 2 profile
17 |   |-- ingestion.py # OCR PDF -> Document
18 |   |-- node_parser.py # Document -> Node theo Điều
19 |   |-- index_builder.py # Node -> VectorStoreIndex (lưu xuống đĩa)
20 |   |-- retriever.py # Retriever top-k + reranker (tùy chọn)
```

```
21 | |-- query_engine.py # Prompt + LLM (Ollama) + trích dẫn Điều  
22 | `-- pipeline.py # Hoi dap qua dong lenh  
23 |-- storage/vector_index/ # Chi muc da lap (528 KB)  
24 `-- requirements.txt
```

Mọi script chạy từ thư mục gốc của repository, ví dụ `RAG_PROFILE=server python src/pipeline.py`. Các module trong `src/` vừa là thư viện (được module sau import) vừa có khối `__main__` để chạy thử riêng từng giai đoạn.

3.4 Cấu hình tập trung và hai profile

Toàn bộ tham số RAG nằm trong `src/config.py` dưới dạng một dataclass `RAGConfig`. Có hai profile sẵn: `local` (máy cá nhân, mô hình nhỏ) và `server` (mô hình lớn hơn, có reranker, LLM chạy trên máy GPU thuê). Profile được chọn bằng biến môi trường `RAG_PROFILE` (đọc cả từ file `.env`), mặc định là `local`; giá trị không hợp lệ làm hàm `get_config()` ném `ValueError`.

Đoạn mã 3.4: Khai báo dataclass `RAGConfig` trong `src/config.py`.

```
1 @dataclass  
2 class RAGConfig:  
3     embed_model_name: str  
4     embed_device: str # "cpu" or "cuda"  
5     llm_model_name: str  
6     ollama_base_url: str  
7     request_timeout: float = 180.0  
8     reranker_model_name: str | None = None  
9     reranker_top_n: int = 3  
10    similarity_top_k: int = 10  
11    response_mode: str = "compact"  
12    streaming: bool = True  
13    persist_dir: str = "storage/vector_index"
```

Tham số	Profile local	Profile server
Embedding	bkai-foundation-models/ vietnamese-bi-encoder (768 chiều)	AITeamVN/Vietnamese_Embedding (1024 chiều)
Thiết bị embedding	cuda	cuda
LLM (Ollama)	qwen3:4b, localhost:11434	qwen3:8b, địa chỉ lấy từ OLLAMA_BASE_URL
Reranker	không dùng	AITeamVN/Vietnamese_Reranker, giữ top 3
Retriever	top-10	top-10

Bảng 3.2: Hai profile cấu hình trong `src/config.py`.

Hai profile dùng chung `persist_dir = "storage/vector_index"`. Vì hai mô hình embedding cho vector khác số chiều, chỉ mục đang lưu (lập bằng profile `server`, 1024 chiều) không dùng được cho profile `local`; muốn chuyển profile phải chạy `index_builder.py --rebuild`, hoặc tách thư mục chỉ mục theo profile (việc nên làm).

Một điểm chưa thống nhất còn lại: trường `corpus_sources` của `RAGConfig` trỏ tới hai file `quy_che_dao_tao_2021.pdf` và `qd_sua_doi_2025.pdf` không tồn tại, và hiện không module nào đọc trường này. Nguồn dữ liệu thực tế được khai báo cứng ở hai nơi: `OCR_FILE` trong `index_builder.py` và `CORPUS_SOURCES` trong `ingestion.py`.

3.5 Phần cứng và bộ nhớ GPU

Máy phát triển dùng card rời NVIDIA RTX A2000 Laptop (4 GB VRAM, khoảng 3,7 GB khả dụng cho PyTorch). Kiểm tra bằng cách nạp mô hình qua đúng hàm của dự án và in thiết bị của các tham số cho thấy cả mô hình embedding lẫn reranker đều nằm trên `cuda:0`. LLM không chạy trên máy này: ở profile `server`, Ollama chạy trên máy GPU thuê từ dịch vụ `ckey.vn`, được quản lý bằng `scripts/manager.py`.

Với mã nguồn hiện tại (nạp mô hình ở độ chính xác `fp32`), profile `server` **không nạp đồng thời** được embedding và reranker trên card 4 GB: khi đó, tiến trình đã chiếm khoảng 3,5 GB và báo `CUDA out of memory` lúc nạp mô hình thứ hai. Để chạy được phép đo có reranker ở Chương 5, embedding được đặt lên CPU và reranker trên

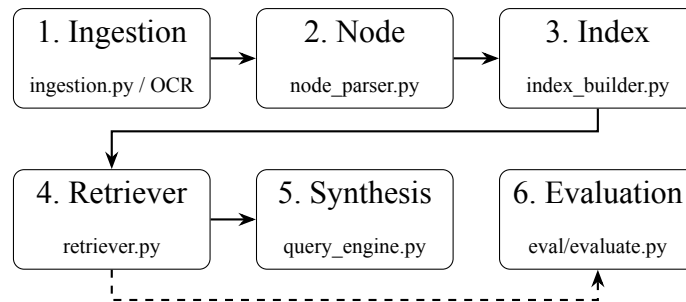
GPU. Một nhánh phát triển khác của dự án đã thử nạp cả hai mô hình ở fp16: embedding còn khoảng 1,1 GB, cả hai cùng nằm trên GPU tổng khoảng 2,2 GB. Thay đổi này chưa được gộp vào nhánh chính nên không trình bày chi tiết ở đây.

3.6 Các script hỗ trợ

- `scripts/test_ocr.py`: render toàn bộ một PDF và OCR ra file Mark-down trong `data/ocr/`, kèm số trang và tổng số ký tự. Đây là script tạo ra bản OCR `data/ocr/4. QuyCheDaoTaoDHSB 2021.md` mà `index_builder.py` dùng làm đầu vào.
- `scripts/check_model_memory.py`: đo RAM/VRAM khi nạp một mô hình embedding hoặc reranker (nhận `repo_id`, `--type`, `--device`), giúp chọn mô hình vừa với phần cứng.
- `scripts/pull_models.sh`: tải LLM qua Ollama và tải embedding/reranker từ HuggingFace theo profile, đồng bộ với `config.py`; dừng với thông báo lỗi nếu thiếu ollama, daemon chưa chạy hoặc thiếu `huggingface_hub`.
- `scripts/manager.py`: công cụ dòng lệnh gom các thao tác với dịch vụ GPU thuê (xem, thuê, khởi động lại, xóa máy; xem số dư), gọi thử Ollama và chạy một server embedding.

3.7 Minh chứng kiến trúc

Chuỗi xử lý của đề án gồm sáu giai đoạn. Hình 3.1 thể hiện chuỗi này và file mã nguồn tương ứng; cả sáu giai đoạn đều đã có mã chạy được. Pha chuẩn bị (1–3) chạy một lần và lưu chỉ mục xuống đĩa; pha truy vấn (4–5) chạy mỗi câu hỏi; giai đoạn 6 dùng lại retriever của giai đoạn 4.



Hình 3.1: Sáu giai đoạn của pipeline RAG và file mã nguồn tương ứng; mũi tên nét đứt: bộ đánh giá gọi trực tiếp retriever.

Giai đoạn	Mức	File	Chạy được
1. Ingestion	Bắt buộc lỗi	src/ingestion.py, scripts/test_ocr.py	Có
2. Node	Bắt buộc lỗi	src/node_parser.py	Có
3. Index	Bắt buộc lỗi	src/index_builder.py	Có
4. Retriever	Bắt buộc lỗi	src/retriever.py	Có
5. Response Synthesis	Bắt buộc lỗi	src/query_engine.py, src/pipeline.py	Có
6. Evaluation	Bắt buộc lỗi	eval/evaluate.py	Có (chỉ truy hồi)
Rerank	Chọn thêm	src/retriever.py	Có (profile server)
Giao diện web	Chọn thêm	app/ (trống)	—

Bảng 3.3: Trạng thái mã nguồn của từng giai đoạn.

Chưa triển khai. Giao diện Gradio (thư mục app/ mới có assets), việc nạp nhiều văn bản vào cùng chỉ mục, và bộ đánh giá tự động cho *câu trả lời* của LLM (hiện chỉ đánh giá truy hồi).

CHƯƠNG 4. BẢN CHẤT CÔNG NGHỆ — BẮT BUỘC LỖI

4.1 Nạp dữ liệu (Ingestion)

4.1.1 Cơ chế

Nhiệm vụ của giai đoạn này là biến file PDF quy chế thành danh sách Document của LlamaIndex. Vì PDF là bản scan nên module `src/ingestion.py` không dùng `SimpleDirectoryReader` để đọc chữ mà đi qua ba bước: (1) render từng trang thành ảnh bằng `pdf2image` ở 300 dpi; (2) OCR từng ảnh bằng `pytesseract` với `lang="vie"`; (3) đóng gói văn bản mỗi trang thành một Document kèm metadata nguồn (tên nguồn), trang (số trang) và `file_goc` (tên file).

Ngoài ra module có:

- lớp ngoại lệ `ScannedPDFLoadError` cho lỗi không tìm thấy file hoặc không render được PDF;
- hàm `has_text_layer()` thử đọc PDF bằng `SimpleDirectoryReader` để biết PDF có text layer không (hàm này định nghĩa sẵn nhưng `load_corpus` hiện chưa gọi tới);
- hàm `load_corpus()` lặp qua danh sách nguồn `CORPUS_SOURCES`, bỏ qua nguồn lỗi và ghi log thay vì dừng toàn bộ chương trình.

4.1.2 Mã nguồn

Trích hàm OCR từng trang (phần xử lý lỗi từng trang là điểm đáng chú ý: một trang lỗi chỉ bị ghi log và thay bằng chuỗi rỗng, không làm hỏng cả file):

Đoạn mã 4.1: Vòng lặp OCR từng trang trong `src/ingestion.py`.

```
1 documents: list[Document] = []
2 failed_pages: list[int] = []
3
4 for i, page in enumerate(pages, start=1):
5     try:
6         text = pytesseract.image_to_string(page, lang=lang)
7     except Exception as e:
8         logger.error(f"OCR lỗi ở trang {i} của {pdf_path}: {e}")
9         failed_pages.append(i)
```

```
10         text = ""
11
12     documents.append(Document(
13         text=text,
14         metadata={"nguồn": source_name, "trang": i, "file_goc":
15                 path.name},
16     ))
```

Hàm `load_corpus` gọi hàm trên cho từng nguồn:

Đoạn mã 4.2: Hàm `load_corpus()` nạp nhiều nguồn PDF.

```
1 def load_corpus(sources: list[tuple[str, str]] | None = None) ->
   list[Document]:
2     sources = sources or CORPUS_SOURCES
3     all_documents: list[Document] = []
4
5     for pdf_path, source_name in sources:
6         try:
7             docs = load_scanned_pdf(pdf_path, source_name)
8             all_documents.extend(docs)
9         except ScannedPDFLoadError as e:
10             logger.error(f"Bỏ qua nguồn '{source_name}' do lỗi: {e}")
11             continue
```

4.1.3 Kết quả chạy thật

Chạy `python src/ingestion.py` với file 4. QuyCheDaoTaoDHSG 2021.pdf mất khoảng 28 giây và cho kết quả:

Đoạn mã 4.3: Kết quả chạy `src/ingestion.py`.

```
1 [INFO] Hoàn tất OCR 4. QuyCheDaoTaoDHSG 2021.pdf: 17 trang, tổng
   34276 ký tự
2 [INFO] Tổng cộng nạp được 17 Document từ 1 nguồn.
```

Tức là 17 trang thành 17 Document, tổng 34 276 ký tự (trung bình khoảng 2 000 ký tự mỗi trang). Ba trang đầu được in ra làm mẫu, ví dụ trang 1 bắt đầu bằng phần quốc hiệu và tiêu đề của quyết định ban hành.

4.1.4 Nhận xét chất lượng OCR

Kiểm tra các trích đoạn in ra cho thấy OCR chưa hoàn hảo. Trang 1 nhận ra “NHÂN DÂN” và “HỒ CHÍ MINH” (đúng phải là “NHÂN DÂN”, “HỒ CHÍ MINH”), còn trang 2 có dòng “THANH PHO HO CHI MINH” mất dấu hoàn toàn. Những lỗi này nằm ở phần quốc hiệu, tiêu đề, không phải phần thân của các Điều, nhưng chúng cho thấy: (1) văn bản sau OCR cần được kiểm tra bằng mắt trước khi tin tưởng; (2) quy tắc cắt theo Điều phải chịu được lỗi OCR ở các từ khóa.

4.1.5 Vị trí trong đồ án

Đây là giai đoạn 1 của pipeline. Đầu ra của `ingestion.py` là danh sách `Document` theo trang, mang metadata nguồn. Đầu ra theo trang có nhược điểm là một Điều nằm vắt qua hai trang sẽ bị chia đôi. Vì vậy, khi lập chỉ mục, hệ thống *không* dùng trực tiếp đầu ra theo trang mà dùng hàm `load_documents()` trong `index_builder.py` (Mục 4.3): hàm này đọc bản OCR đã lưu, nối toàn bộ các trang thành một `Document` duy nhất, và chỉ gọi lại `ingestion.py` để OCR từ PDF khi chưa có bản OCR.

4.2 Cắt đoạn theo Điều (Node)

4.2.1 Cơ chế

Module `src/node_parser.py` chuyển `Document` thành `TextNode` theo đơn vị Điều. Hàm `parse_by_dieu()` làm như sau:

1. Dùng biểu thức chính quy `DIEU_PATTERN` để tìm từng đoạn bắt đầu bằng “Điều *N*.” hoặc “Điều *N*:” và kéo dài tới ngay trước Điều kế tiếp (hoặc hết văn bản).
2. Với mỗi đoạn, trích số Điều bằng `extract_dieu_number()` và gắn vào `metadata['dieu']`, giữ nguyên metadata gốc của `Document`.
3. Nếu số từ của Điều (đếm gần đúng bằng tách theo khoảng trắng) vượt `max_chunk_tokens` (mặc định 1024), cắt tiếp bằng `SentenceSplitter` với `chunk_overlap = 50` và gắn thêm `metadata['phan']`.
4. `Document` không khớp mẫu nào được giữ nguyên thành một `Node` với `dieu = "khong_co_dieu"`, đồng thời ghi cảnh báo.

Hàm `validate_nodes()` thống kê chất lượng: tổng số Node, số Node không rõ Điều, tỷ lệ và độ dài trung bình.

4.2.2 Mã nguồn

Biểu thức chính quy và hàm trích số Điều:

Đoạn mã 4.4: Biểu thức chính quy tách Điều và hàm trích số Điều.

```
1 DIEU_PATTERN =  
    re.compile(r"(Điều\s+\d+[.:.]*?)(?=Điều\s+\d+[.:.]|\Z)",  
    re.DOTALL)  
2 DIEU_NUMBER_PATTERN = re.compile(r"Điều\s+(\d+)")  
3  
4 def extract_dieu_number(text: str) -> str:  
5     """Trích xuất số thứ tự của Điều từ đoạn văn bản."""  
6     match = DIEU_NUMBER_PATTERN.match(text.strip())  
7     return match.group(1) if match else "khong_ro"
```

Giải thích biểu thức: `Điều\s+\d+[.:.]` khớp phần đầu Điều (từ “Điều”, khoảng trắng, chữ số, rồi dấu chấm hoặc hai chấm); `. * ?` lấy nhỏ nhất có thể (lazy) với cờ `DOTALL` để qua nhiều dòng; `(?=...|\Z)` là lookahead dừng ngay trước Điều kế tiếp hoặc cuối văn bản. Yêu cầu có dấu chấm/hai chấm sau số là điều kiện quan trọng: nhờ đó các tham chiếu chéo kiểu “theo quy định tại Điều 5 của Quy chế này” không bị nhầm thành tiêu đề Điều mới.

Phần lõi của hàm cắt:

Đoạn mã 4.5: Vòng lặp cắt Node theo Điều trong `parse_by_dieu()`.

```
1 for dieu_text in matches:  
2     dieu_so = extract_dieu_number(dieu_text)  
3     base_metadata = {"dieu": dieu_so, **doc.metadata}  
4     estimated_tokens = len(dieu_text.split())  
5  
6     if estimated_tokens <= max_chunk_tokens:  
7         all_nodes.append(TextNode(text=dieu_text.strip(),  
            metadata=base_metadata))  
8     else:  
9         sub_texts = sub_splitter.split_text(dieu_text)  
10        for j, sub_text in enumerate(sub_texts):  
11            all_nodes.append(  
12                TextNode(  

```

```
13         text=sub_text,  
14         metadata={**base_metadata, "phan": j + 1},  
15     )  
16 )
```

4.2.3 Kết quả chạy thật

Khởi `__main__` của `node_parser.py` đọc bản OCR đã lưu `data/ocr/4.QuyCheDaoTaoDHSB 2021.md` (không phải đầu ra trực tiếp của `ingestion.py`), bọc thành một `Document` rồi cắt. Kết quả chạy thật:

Đoạn mã 4.6: Kết quả chạy `node_parser.py`.

```
1 - tong_so_node: 24  
2 - node_khong_ro_dieu: 0  
3 - ty_le_khong_ro: 0.0%  
4 - do_dai_trung_binh_ky_tu: 1400  
5  
6 [Node 1] - Metadata: {'dieu': '1', 'source': '4. QuyCheDaoTaoDHSB  
    2021.md', 'doc_type': 'quy_che'}  
7 [Node 2] - Metadata: {'dieu': '2', ...}  
8 [Node 3] - Metadata: {'dieu': '3', ...}
```

Có 24 Node, không Node nào bị gán “không rõ Điều” (0%), độ dài trung bình 1 400 ký tự.

4.2.4 Phân tích kết quả

Kiểm tra thêm bằng cách in danh sách nhãn `dieu` của toàn bộ 24 Node cho thấy:

- Văn bản OCR gồm hai phần nối nhau: *Quyết định ban hành* (3 Điều) và *Quy chế* (21 Điều). Do đó chuỗi nhãn là 1, 2, 3 rồi 1, 2, ..., 21: **ba nhãn Điều 1, 2, 3 xuất hiện hai lần** (một lần của Quyết định, một lần của Quy chế).
- Trong toàn bộ 24 Node không có Node nào bị cắt nhỏ (không có `phan`), tức không Điều nào vượt ngưỡng 1024 từ. Điều dài nhất khoảng 3 238 ký tự.
- Có 36 lần xuất hiện cụm “Điều *N*” trong văn bản nhưng chỉ 24 lần là tiêu đề Điều; 12 lần còn lại chủ yếu là tham chiếu chéo (không có dấu chấm sau số) và đã không bị tách nhầm, đúng như thiết kế.

Hệ quả và cách đã xử lý: vì hai Điều mang cùng nhãn (ví dụ Điều 2 của Quyết định và Điều 2 của Quy chế) nên trường `diều` một mình không phân biệt được chúng; khi chấm truy hồi bằng `diều_dung`, Node của Quyết định ban hành có thể bị tính nhầm là đúng. Do Quyết định ban hành nằm trọn ở trang 1 của bản OCR và không chứa quy định nào cho sinh viên, hàm `load_documents()` của bước lập chỉ mục bỏ trang 1 trước khi cắt. Kết quả là chỉ mục chỉ còn đúng 21 Điều của Quy chế, số Điều trong metadata khớp với số Điều trong văn bản (Mục 4.3). Cách này chỉ đúng cho văn bản hiện tại; khi nạp thêm văn bản, cần thêm metadata tên văn bản vào Node.

4.2.5 Hạn chế đã biết

- Số “token” được đếm bằng số từ tách theo khoảng trắng, chỉ là xấp xỉ, không phải token của mô hình.
- Nếu OCR làm mất dấu chấm sau số Điều (ví dụ “Điều 12” không có dấu chấm) thì tiêu đề đó không khớp mẫu và Điều bị gộp vào Điều trước.
- Ngược lại, một câu tham chiếu kết thúc bằng dấu chấm ngay sau số Điều (“...quy định tại Điều 5.”) có thể bị nhận nhầm là tiêu đề mới. Trong văn bản hiện có, trường hợp này chưa xảy ra.
- Kiểm thử mới chạy trên một văn bản; chưa có kiểm thử tự động.
- Khối `__main__` của `node_parser.py` vẫn đọc cả file OCR (24 Node) để minh họa, trong khi pipeline thật đi qua `load_documents()` (21 Node).

4.2.6 Vị trí trong đồ án

Đây là giai đoạn 2 của pipeline. Mỗi Node là một Điều với nhãn `diều`, làm đơn vị cơ bản để lập chỉ mục và để trích dẫn nguồn khi trả lời.

4.3 Lập chỉ mục (Index)

4.3.1 Cơ chế

Module `src/index_builder.py` làm ba việc:

1. **Nạp văn bản** (`load_documents()`): đọc bản OCR `data/ocr/4.QuyCheDaoTaoDHSg 2021.md`, cắt bỏ phần trước “## Trang 2”

(Quyết định ban hành), xóa các dòng tiêu đề trang “## Trang N” và dòng phân cách --- do `test_ocr.py` chèn vào, rồi trả về *một* Document với metadata nguồn và `file_goc`. Nhờ nối các trang, không Điều nào bị chia đôi ở ranh giới trang.

2. **Đặt mô hình embedding** (`setup_embedding()`): gán `Settings.embed_model` bằng `HuggingFaceEmbedding` theo profile. `Settings` là cấu hình toàn cục của `LlamaIndex`: mọi thành phần cần embedding (lập chỉ mục, retriever) tự lấy mô hình từ đây.
3. **Lập hoặc nạp lại chỉ mục** (`build_or_load_index()`): nếu thư mục `persist_dir` đã có `docstore.json` thì nạp chỉ mục từ đĩa, không phải embedding lại; ngược lại (hoặc khi chạy với `--rebuild`) thì cắt Node bằng `parse_by_dieu()`, dựng `VectorStoreIndex` và lưu xuống đĩa.

4.3.2 Mã nguồn

Đoạn mã 4.7: Lập hoặc nạp lại chỉ mục trong `src/index_builder.py`.

```
1 def setup_embedding(cfg: RAGConfig) -> None:
2     Settings.embed_model =
3         HuggingFaceEmbedding(model_name=cfg.embed_model_name,
4                               device=cfg.embed_device)
5
6 def build_or_load_index(cfg: RAGConfig, rebuild: bool = False) ->
7     VectorStoreIndex:
8     setup_embedding(cfg)
9     persist_dir = ROOT / cfg.persist_dir
10
11     if not rebuild and (persist_dir / "docstore.json").exists():
12         logger.info(f"Load Index từ {persist_dir}")
13         return
14         load_index_from_storage(StorageContext.from_defaults(persist_dir=str(persist_dir)))
15
16 nodes = parse_by_dieu(load_documents())
17 logger.info(f"Build Index mới
18           từ {len(nodes)} node: {validate_nodes(nodes)}")
19 index = VectorStoreIndex(nodes, show_progress=True)
20 persist_dir.mkdir(parents=True, exist_ok=True)
21 index.storage_context.persist(persist_dir=str(persist_dir))
```

Điểm đáng chú ý: `VectorStoreIndex` được dựng thẳng từ danh sách `TextNode` đã cắt sẵn chứ không từ `Document`. Nếu truyền `Document`, `LlamaIndex` sẽ tự cắt bằng `SentenceSplitter` mặc định và làm mất ranh giới Điều. Hàm `setup_embedding()` được gọi *cả* khi nạp lại chỉ mục, vì câu hỏi lúc truy vấn vẫn phải được embedding bằng đúng mô hình đã dùng khi lập chỉ mục.

4.3.3 Kết quả chạy thật

Với profile server, nạp Node qua `load_documents()` cho:

Đoạn mã 4.8: Thống kê Node đưa vào chỉ mục.

```
1 tong_so_node: 21 | node_khong_ro_dieu: 0 | ty_le_khong_ro: 0.0% |  
   do_dai_trung_binh_ky_tu: 1538  
2 dieu: 1, 2, 3, ..., 21 (moi Dieu xuất hiện đúng 1 lần)
```

Chỉ mục lưu tại `storage/vector_index` gồm 21 vector 1024 chiều (mô hình `AITeamVN/Vietnamese_Embedding`), tổng dung lượng 528 KB, trong đó `default__vector_store.json` (vector) 436 KB và `docstore.json` (nội dung và metadata Node) 85 KB. Chạy `python src/index_builder.py` ở chế độ nạp lại in ra:

Đoạn mã 4.9: Kết quả chạy `src/index_builder.py`.

```
1 [INFO] Load Index từ .../storage/vector_index  
2 Số node trong Index: 21 | số node từ node_parser: 21 | khớp: True
```

Dòng cuối là phép kiểm tra nhất quán: số Node trong chỉ mục đã lưu bằng số Node mà `node_parser` cắt ra từ dữ liệu hiện tại, tức chỉ mục không bị cũ so với dữ liệu. So với Mục trước (24 Node khi cắt cả file), ba Node của Quyết định ban hành đã bị loại và không còn nhãn Điều trùng. Điều dài nhất 3 180 ký tự, không Điều nào vượt ngưỡng phải cắt nhỏ. Thời gian nạp lại (gồm nạp mô hình embedding lên GPU) khoảng 15–18 giây.

4.3.4 Vị trí trong đồ án

Đây là giai đoạn 3, ranh giới giữa pha chuẩn bị và pha truy vấn. Nhờ lưu xuống đĩa, các lần chạy sau (hỏi đáp, đánh giá) chỉ nạp lại chỉ mục mà không phải OCR hay embedding

lại.

4.4 Truy hồi (Retriever) và xếp hạng lại

4.4.1 Cơ chế

Module `src/retriever.py` có hai hàm:

- `get_retriever()`: gọi `index.as_retriever(similarity_top_k=10)`. Retriever embedding câu hỏi bằng `Settings.embed_model`, tính cosine với 21 vector trong chỉ mục và trả về 10 Node điểm cao nhất (`NodeWithScore`).
- `get_postprocessors()`: trả về danh sách rỗng nếu profile không đặt reranker; ngược lại trả về một `SentenceTransformerRerank` chấm lại 10 Node bằng cross-encoder và giữ 3 Node đầu.

4.4.2 Mã nguồn

Đoạn mã 4.10: Retriever và reranker trong `src/retriever.py`.

```
1 def get_retriever(index: VectorStoreIndex, cfg: RAGConfig) ->
   BaseRetriever:
2     return index.as_retriever(similarity_top_k=cfg.similarity_top_k)
3
4
5 def get_postprocessors(cfg: RAGConfig) -> list:
6     """Rerank chỉ bật khi config đặt reranker_model_name."""
7     if not cfg.reranker_model_name:
8         return []
9     from llama_index.core.postprocessor import
       SentenceTransformerRerank
10    return [SentenceTransformerRerank(model=cfg.reranker_model_name,
        top_n=cfg.reranker_top_n)]
```

Reranker được trả về dưới dạng *node postprocessor* chứ không gộp vào retriever: `RetrieverQueryEngine` gọi retriever trước, rồi lần lượt đưa kết quả qua các post-processor. Nhờ tách như vậy, bộ đánh giá (Chương 5) có thể đo riêng retriever, và việc bật/tắt reranker chỉ phụ thuộc vào cấu hình.

4.4.3 Kết quả chạy thật

Chạy python `src/retriever.py` (profile server, không qua reranker) với câu hỏi mẫu “Một năm học có bao nhiêu học kỳ?”:

Đoạn mã 4.11: Kết quả chạy `src/retriever.py` (rút gọn nội dung Node).

```
1 [1] score=0.4470 | Điều 6 | 'Điều 6. Kế hoạch giảng dạy và học tập  
...'  
2 [2] score=0.3872 | Điều 10 | 'Điều 10. Đánh giá kết quả học tập theo  
học kì, năm học ...'  
3 [3] score=0.3763 | Điều 7 | 'Điều 7. Tổ chức đăng ký học tập ...'  
4 [4] score=0.3140 | Điều 9 | 'Điều 9. Đánh giá và tính điểm học phần  
...'  
5 ...  
6 [10] score=0.2578 | Điều 17 | 'Điều 17. Học cùng lúc hai chương  
trình ...'
```

Điều đúng (Điều 6, khoản 2: “mỗi năm học có 02 học kì chính ... và 01 học kì phụ”) đúng đầu. Hai nhận xét: (1) điểm cosine tuyệt đối khá thấp (0,45) và cách biệt giữa hạng 1 và hạng 10 chỉ 0,19, vì mỗi Node là cả một Điều dài khoảng 1 500 ký tự trong khi đáp án chỉ là một câu trong đó; (2) cùng lý do, nên lấy k đủ rộng rồi để LLM (hoặc reranker) chọn, thay vì chỉ lấy Node đầu.

4.5 Tổng hợp câu trả lời (Query Engine)

4.5.1 Cơ chế

Module `src/query_engine.py` ghép retriever, reranker và LLM thành một `RetrieverQueryEngine`:

- **LLM**: `Ollama(model=..., base_url=..., thinking=False, context_window=8192)`. Tắt chế độ suy nghĩ của Qwen3 để câu trả lời ngắn; cửa sổ ngữ cảnh 8 192 token đủ chứa 10 Điều.
- **Prompt riêng** (`QA_PROMPT`) thay cho prompt tiếng Anh mặc định của `LlamaIndex`: nêu rõ ngữ cảnh là trích đoạn Quy chế SGU, yêu cầu *chỉ* dựa vào trích đoạn, trả lời bằng tiếng Việt ngắn gọn, và nói “không tìm thấy trong quy chế” khi trích đoạn không có thông tin. Câu cuối là cơ chế chống bịa: cho mô hình một lối ra hợp lệ thay vì phải đoán.
- **Trích dẫn nguồn**: `format_sources()` đọc `metadata['dieu']` của các

Node nguồn trong `response.source_nodes` và in “Nguồn: Điều X, Điều Y...”. Nguồn được lấy từ metadata chứ không nhờ LLM tự ghi, nên luôn đúng với những gì đã đưa vào prompt.

- **Streaming:** khi `streaming=True`, câu trả lời được in dần từng token qua `response.response_gen`.

`src/pipeline.py` là vòng lặp hỏi đáp trên dòng lệnh: dựng chỉ mục và query engine một lần, sau đó đọc câu hỏi, gọi `ask()`, cho tới khi gõ thoát.

4.5.2 Mã nguồn

Đoạn mã 4.12: Prompt và hàm dựng query engine trong `src/query_engine.py`.

```
1 QA_PROMPT = PromptTemplate(  
2     "Dưới đây là các trích đoạn từ Quy chế đào tạo trình độ đại học  
3     Trường Đại học Sài Gòn.\n"  
4     "-----\n{context_str}\n-----\n"  
5     "Chỉ dựa vào các trích đoạn trên, trả lời câu hỏi bằng tiếng  
6     Việt, ngắn gọn và chính xác. "  
7     "Nếu trích đoạn không có thông tin, hãy nói không tìm thấy trong  
8     quy chế.\n"  
9     "Câu hỏi: {query_str}\n"  
10    "Trả lời: "  
11 )  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132  
133  
134  
135  
136  
137  
138  
139  
140  
141  
142  
143  
144  
145  
146  
147  
148  
149  
150  
151  
152  
153  
154  
155  
156  
157  
158  
159  
160  
161  
162  
163  
164  
165  
166  
167  
168  
169  
170  
171  
172  
173  
174  
175  
176  
177  
178  
179  
180  
181  
182  
183  
184  
185  
186  
187  
188  
189  
190  
191  
192  
193  
194  
195  
196  
197  
198  
199  
200  
201  
202  
203  
204  
205  
206  
207  
208  
209  
210  
211  
212  
213  
214  
215  
216  
217  
218  
219  
220  
221  
222  
223  
224  
225  
226  
227  
228  
229  
230  
231  
232  
233  
234  
235  
236  
237  
238  
239  
240  
241  
242  
243  
244  
245  
246  
247  
248  
249  
250  
251  
252  
253  
254  
255  
256  
257  
258  
259  
260  
261  
262  
263  
264  
265  
266  
267  
268  
269  
270  
271  
272  
273  
274  
275  
276  
277  
278  
279  
280  
281  
282  
283  
284  
285  
286  
287  
288  
289  
290  
291  
292  
293  
294  
295  
296  
297  
298  
299  
300  
301  
302  
303  
304  
305  
306  
307  
308  
309  
310  
311  
312  
313  
314  
315  
316  
317  
318  
319  
320  
321  
322  
323  
324  
325  
326  
327  
328  
329  
330  
331  
332  
333  
334  
335  
336  
337  
338  
339  
340  
341  
342  
343  
344  
345  
346  
347  
348  
349  
350  
351  
352  
353  
354  
355  
356  
357  
358  
359  
360  
361  
362  
363  
364  
365  
366  
367  
368  
369  
370  
371  
372  
373  
374  
375  
376  
377  
378  
379  
380  
381  
382  
383  
384  
385  
386  
387  
388  
389  
390  
391  
392  
393  
394  
395  
396  
397  
398  
399  
400  
401  
402  
403  
404  
405  
406  
407  
408  
409  
410  
411  
412  
413  
414  
415  
416  
417  
418  
419  
420  
421  
422  
423  
424  
425  
426  
427  
428  
429  
430  
431  
432  
433  
434  
435  
436  
437  
438  
439  
440  
441  
442  
443  
444  
445  
446  
447  
448  
449  
450  
451  
452  
453  
454  
455  
456  
457  
458  
459  
460  
461  
462  
463  
464  
465  
466  
467  
468  
469  
470  
471  
472  
473  
474  
475  
476  
477  
478  
479  
480  
481  
482  
483  
484  
485  
486  
487  
488  
489  
490  
491  
492  
493  
494  
495  
496  
497  
498  
499  
500  
501  
502  
503  
504  
505  
506  
507  
508  
509  
510  
511  
512  
513  
514  
515  
516  
517  
518  
519  
520  
521  
522  
523  
524  
525  
526  
527  
528  
529  
530  
531  
532  
533  
534  
535  
536  
537  
538  
539  
540  
541  
542  
543  
544  
545  
546  
547  
548  
549  
550  
551  
552  
553  
554  
555  
556  
557  
558  
559  
560  
561  
562  
563  
564  
565  
566  
567  
568  
569  
570  
571  
572  
573  
574  
575  
576  
577  
578  
579  
580  
581  
582  
583  
584  
585  
586  
587  
588  
589  
590  
591  
592  
593  
594  
595  
596  
597  
598  
599  
600  
601  
602  
603  
604  
605  
606  
607  
608  
609  
610  
611  
612  
613  
614  
615  
616  
617  
618  
619  
620  
621  
622  
623  
624  
625  
626  
627  
628  
629  
630  
631  
632  
633  
634  
635  
636  
637  
638  
639  
640  
641  
642  
643  
644  
645  
646  
647  
648  
649  
650  
651  
652  
653  
654  
655  
656  
657  
658  
659  
660  
661  
662  
663  
664  
665  
666  
667  
668  
669  
670  
671  
672  
673  
674  
675  
676  
677  
678  
679  
680  
681  
682  
683  
684  
685  
686  
687  
688  
689  
690  
691  
692  
693  
694  
695  
696  
697  
698  
699  
700  
701  
702  
703  
704  
705  
706  
707  
708  
709  
710  
711  
712  
713  
714  
715  
716  
717  
718  
719  
720  
721  
722  
723  
724  
725  
726  
727  
728  
729  
730  
731  
732  
733  
734  
735  
736  
737  
738  
739  
740  
741  
742  
743  
744  
745  
746  
747  
748  
749  
750  
751  
752  
753  
754  
755  
756  
757  
758  
759  
760  
761  
762  
763  
764  
765  
766  
767  
768  
769  
770  
771  
772  
773  
774  
775  
776  
777  
778  
779  
780  
781  
782  
783  
784  
785  
786  
787  
788  
789  
790  
791  
792  
793  
794  
795  
796  
797  
798  
799  
800  
801  
802  
803  
804  
805  
806  
807  
808  
809  
810  
811  
812  
813  
814  
815  
816  
817  
818  
819  
820  
821  
822  
823  
824  
825  
826  
827  
828  
829  
830  
831  
832  
833  
834  
835  
836  
837  
838  
839  
840  
841  
842  
843  
844  
845  
846  
847  
848  
849  
850  
851  
852  
853  
854  
855  
856  
857  
858  
859  
860  
861  
862  
863  
864  
865  
866  
867  
868  
869  
870  
871  
872  
873  
874  
875  
876  
877  
878  
879  
880  
881  
882  
883  
884  
885  
886  
887  
888  
889  
890  
891  
892  
893  
894  
895  
896  
897  
898  
899  
900  
901  
902  
903  
904  
905  
906  
907  
908  
909  
910  
911  
912  
913  
914  
915  
916  
917  
918  
919  
920  
921  
922  
923  
924  
925  
926  
927  
928  
929  
930  
931  
932  
933  
934  
935  
936  
937  
938  
939  
940  
941  
942  
943  
944  
945  
946  
947  
948  
949  
950  
951  
952  
953  
954  
955  
956  
957  
958  
959  
960  
961  
962  
963  
964  
965  
966  
967  
968  
969  
970  
971  
972  
973  
974  
975  
976  
977  
978  
979  
980  
981  
982  
983  
984  
985  
986  
987  
988  
989  
990  
991  
992  
993  
994  
995  
996  
997  
998  
999  
1000  
1001  
1002  
1003  
1004  
1005  
1006  
1007  
1008  
1009  
1010  
1011  
1012  
1013  
1014  
1015  
1016  
1017  
1018  
1019  
1020  
1021  
1022  
1023  
1024  
1025  
1026  
1027  
1028  
1029  
1030  
1031  
1032  
1033  
1034  
1035  
1036  
1037  
1038  
1039  
1040  
1041  
1042  
1043  
1044  
1045  
1046  
1047  
1048  
1049  
1050  
1051  
1052  
1053  
1054  
1055  
1056  
1057  
1058  
1059  
1060  
1061  
1062  
1063  
1064  
1065  
1066  
1067  
1068  
1069  
1070  
1071  
1072  
1073  
1074  
1075  
1076  
1077  
1078  
1079  
1080  
1081  
1082  
1083  
1084  
1085  
1086  
1087  
1088  
1089  
1090  
1091  
1092  
1093  
1094  
1095  
1096  
1097  
1098  
1099  
1100  
1101  
1102  
1103  
1104  
1105  
1106  
1107  
1108  
1109  
1110  
1111  
1112  
1113  
1114  
1115  
1116  
1117  
1118  
1119  
1120  
1121  
1122  
1123  
1124  
1125  
1126  
1127  
1128  
1129  
1130  
1131  
1132  
1133  
1134  
1135  
1136  
1137  
1138  
1139  
1140  
1141  
1142  
1143  
1144  
1145  
1146  
1147  
1148  
1149  
1150  
1151  
1152  
1153  
1154  
1155  
1156  
1157  
1158  
1159  
1160  
1161  
1162  
1163  
1164  
1165  
1166  
1167  
1168  
1169  
1170  
1171  
1172  
1173  
1174  
1175  
1176  
1177  
1178  
1179  
1180  
1181  
1182  
1183  
1184  
1185  
1186  
1187  
1188  
1189  
1190  
1191  
1192  
1193  
1194  
1195  
1196  
1197  
1198  
1199  
1200  
1201  
1202  
1203  
1204  
1205  
1206  
1207  
1208  
1209  
1210  
1211  
1212  
1213  
1214  
1215  
1216  
1217  
1218  
1219  
1220  
1221  
1222  
1223  
1224  
1225  
1226  
1227  
1228  
1229  
1230  
1231  
1232  
1233  
1234  
1235  
1236  
1237  
1238  
1239  
1240  
1241  
1242  
1243  
1244  
1245  
1246  
1247  
1248  
1249  
1250  
1251  
1252  
1253  
1254  
1255  
1256  
1257  
1258  
1259  
1260  
1261  
1262  
1263  
1264  
1265  
1266  
1267  
1268  
1269  
1270  
1271  
1272  
1273  
1274  
1275  
1276  
1277  
1278  
1279  
1280  
1281  
1282  
1283  
1284  
1285  
1286  
1287  
1288  
1289  
1290  
1291  
1292  
1293  
1294  
1295  
1296  
1297  
1298  
1299  
1300  
1301  
1302  
1303  
1304  
1305  
1306  
1307  
1308  
1309  
1310  
1311  
1312  
1313  
1314  
1315  
1316  
1317  
1318  
1319  
1320  
1321  
1322  
1323  
1324  
1325  
1326  
1327  
1328  
1329  
1330  
1331  
1332  
1333  
1334  
1335  
1336  
1337  
1338  
1339  
1340  
1341  
1342  
1343  
1344  
1345  
1346  
1347  
1348  
1349  
1350  
1351  
1352  
1353  
1354  
1355  
1356  
1357  
1358  
1359  
1360  
1361  
1362  
1363  
1364  
1365  
1366  
1367  
1368  
1369  
1370  
1371  
1372  
1373  
1374  
1375  
1376  
1377  
1378  
1379  
1380  
1381  
1382  
1383  
1384  
1385  
1386  
1387  
1388  
1389  
1390  
1391  
1392  
1393  
1394  
1395  
1396  
1397  
1398  
1399  
1400  
1401  
1402  
1403  
1404  
1405  
1406  
1407  
1408  
1409  
1410  
1411  
1412  
1413  
1414  
1415  
1416  
1417  
1418  
1419  
1420  
1421  
1422  
1423  
1424  
1425  
1426  
1427  
1428  
1429  
1430  
1431  
1432  
1433  
1434  
1435  
1436  
1437  
1438  
1439  
1440  
1441  
1442  
1443  
1444  
1445  
1446  
1447  
1448  
1449  
1450  
1451  
1452  
1453  
1454  
1455  
1456  
1457  
1458  
1459  
1460  
1461  
1462  
1463  
1464  
1465  
1466  
1467  
1468  
1469  
1470  
1471  
1472  
1473  
1474  
1475  
1476  
1477  
1478  
1479  
1480  
1481  
1482  
1483  
1484  
1485  
1486  
1487  
1488  
1489  
1490  
1491  
1492  
1493  
1494  
1495  
1496  
1497  
1498  
1499  
1500  
1501  
1502  
1503  
1504  
1505  
1506  
1507  
1508  
1509  
1510  
1511  
1512  
1513  
1514  
1515  
1516  
1517  
1518  
1519  
1520  
1521  
1522  
1523  
1524  
1525  
1526  
1527  
1528  
1529  
1530  
1531  
1532  
1533  
1534  
1535  
1536  
1537  
1538  
1539  
1540  
1541  
1542  
1543  
1544  
1545  
1546  
1547  
1548  
1549  
1550  
1551  
1552  
1553  
1554  
1555  
1556  
1557  
1558  
1559  
1560  
1561  
1562  
1563  
1564  
1565  
1566  
1567  
1568  
1569  
1570  
1571  
1572  
1573  
1574  
1575  
1576  
1577  
1578  
1579  
1580  
1581  
1582  
1583  
1584  
1585  
1586  
1587  
1588  
1589  
1590  
1591  
1592  
1593  
1594  
1595  
1596  
1597  
1598  
1599  
1600  
1601  
1602  
1603  
1604  
1605  
1606  
1607  
1608  
1609  
1610  
1611  
1612  
1613  
1614  
1615  
1616  
1617  
1618  
1619  
1620  
1621  
1622  
1623  
1624  
1625  
1626  
1627  
1628  
1629  
1630  
1631  
1632  
1633  
1634  
1635  
1636  
1637  
1638  
1639  
1640  
1641  
1642  
1643  
1644  
1645  
1646  
1647  
1648  
1649  
1650  
1651  
1652  
1653  
1654  
1655  
1656  
1657  
1658  
1659  
1660  
1661  
1662  
1663  
1664  
1665  
1666  
1667  
1668  
1669  
1670  
1671  
1672  
1673  
1674  
1675  
1676  
1677  
1678  
1679  
1680  
1681  
1682  
1683  
1684  
1685  
1686  
1687  
1688  
1689  
1690  
1691  
1692  
1693  
1694  
1695  
1696  
1697  
1698  
1699  
1700  
1701  
1702  
1703  
1704  
1705  
1706  
1707  
1708  
1709  
1710  
1711  
1712  
1713  
1714  
1715  
1716  
1717  
1718  
1719  
1720  
1721  
1722  
1723  
1724  
1725  
1726  
1727  
1728  
1729  
1730  
1731  
1732  
1733  
1734  
1735  
1736  
1737  
1738  
1739  
1740  
1741  
1742  
1743  
1744  
1745  
1746  
1747  
1748  
1749  
1750  
1751  
1752  
1753  
1754  
1755  
1756  
1757  
1758  
1759  
1760  
1761  
1762  
1763  
1764  
1765  
1766  
1767  
1768  
1769  
1770  
1771  
1772  
1773  
1774  
1775  
1776  
1777  
1778  
1779  
1780  
1781  
1782  
1783  
1784  
1785  
1786  
1787  
1788  
1789  
1790  
1791  
1792  
1793  
1794  
1795  
1796  
1797  
1798  
1799  
1800  
1801  
1802  
1803  
1804  
1805  
1806  
1807  
1808  
1809  
1810  
1811  
1812  
1813  
1814  
1815  
1816  
1817  
1818  
1819  
1820  
1821  
1822  
1823  
1824  
1825  
1826  
1827  
1828  
1829  
1830  
1831  
1832  
1833  
1834  
1835  
1836  
1837  
1838  
1839  
1840  
1841  
1842  
1843  
1844  
1845  
1846  
1847  
1848  
1849  
1850  
1851  
1852  
1853  
1854  
1855  
1856  
1857  
1858  
1859  
1860  
1861  
1862  
1863  
1864  
1865  
1866  
1867  
1868  
1869  
1870  
1871  
1872  
1873  
1874  
1875  
1876  
1877  
1878  
1879  
1880  
1881  
1882  
1883  
1884  
1885  
1886  
1887  
1888  
1889  
1890  
1891  
1892  
1893  
1894  
1895  
1896  
1897  
1898  
1899  
1900  
1901  
1902  
1903  
1904  
1905  
1906  
1907  
1908  
1909  
1910  
1911  
1912  
1913  
1914  
1915  
1916  
1917  
1918  
1919  
1920  
1921  
1922  
1923  
1924  
1925  
1926  
1927  
1928  
1929  
1930  
1931  
1932  
1933  
1934  
1935  
1936  
1937  
1938  
1939  
1940  
1941  
1942  
1943  
1944  
1945  
1946  
1947  
1948  
1949  
1950  
1951  
1952  
1953  
1954  
1955  
1956  
1957  
1958  
1959  
1960  
1961  
1962  
1963  
1964  
1965  
1966  
1967  
1968  
1969  
1970  
1971  
1972  
1973  
1974  
1975  
1976  
1977  
1978  
1979  
1980  
1981  
1982  
1983  
1984  
1985  
1986  
1987  
1988  
1989  
1990  
1991  
1992  
1993  
1994  
1995  
1996  
1997  
1998  
1999  
2000  
2001  
2002  
2003  
2004  
2005  
2006  
2007  
2008  
2009  
2010  
2011  
2012  
2013  
2014  
2015  
2016  
2017  
2018  
2019  
2020  
2021  
2022  
2023  
2024  
2025  
2026  
2027  
2028  
2029  
2030  
2031  
2032  
2033  
2034  
2035  
2036  
2037  
2038  
2039  
2040  
2041  
2042  
2043  
2044  
2045  
2046  
2047  
2048  
2049  
2050  
2051  
2052  
2053  
2054  
2055  
2056  
2057  
2058  
2059  
2060  
2061  
2062  
2063  
2064  
2065  
2066  
2067  
2068  
2069  
2070  
2071  
2072  
2073  
2074  
2075  
2076  
2077  
2078  
2079  
2080  
2081  
2082  
2083  
2084  
2085  
2086  
2087  
2088  
2089  
2090  
2091  
2092  
2093  
2094  
2095  
2096  
2097  
2098  
2099  
2100  
2101  
2102  
2103  
2104  
2105  
2106  
2107  
2108  
2109  
2110  
2111  
2112  
2113  
2114  
2115  
2116  
2117  
2118  
2119  
2120  
2121  
2122  
2123  
2124  
2125  
2126  
2127  
2128  
2129  
2130  
2131  
2132  
2133  
2134  
2135  
2136  
2137  
2138  
2139  
2140  
2141  
2142  
2143  
2144  
2145  
2146  
2147  
2148  
2149  
2150  
2151  
2152
```

```
26     node_postprocessors=get_postprocessors(cfg),  
27 )
```

4.5.3 Kết quả chạy thật

Nhóm đã chạy `src/pipeline.py` ở profile server (embedding và reranker trên máy cá nhân, qwen3:8b trên máy GPU thuê) và hỏi 56 câu tự do, nhiều câu viết tắt, sai chính tả hoặc ngoài phạm vi quy chế. Nhật ký phiên được lưu ở `eval/results/test-basic.md`. Một số trao đổi tiêu biểu:

Đoạn mã 4.13: Trích nhật ký chạy `src/pipeline.py` (profile server).

```
1 Câu hỏi: tôi học cử nhân thì cần học bao nhiêu chỉ  
2 Theo quy chế, chương trình đào tạo cấp bằng cử nhân yêu cầu người  
   học tích lũy tối thiểu 120 tín chỉ.  
3 Nguồn: Điều 2, Điều 10, Điều 9  
4  
5 Câu hỏi: kỳ 1 tôi bị cảnh báo, kỳ 2 bình thường, kỳ 3 bị cảnh báo  
   thì có bị buộc thôi học không ?  
6 Theo quy chế, người học bị buộc thôi học nếu bị cảnh báo liên tiếp  
   quá hai lần. Trong trường hợp  
7 của bạn ... đây là hai lần cảnh báo không liên tiếp, do đó không bị  
   buộc thôi học.  
8 Nguồn: Điều 11, Điều 14, Điều 19  
9  
10 Câu hỏi: Có những quy định nào về điểm rèn luyện  
11 Không tìm thấy trong quy chế.  
12 Nguồn: Điều 3, Điều 12, Điều 2
```

Câu thứ nhất và thứ hai đúng và có trích dẫn đúng Điều. Câu thứ hai đáng chú ý vì mô hình phải *suy luận* trên quy định (phân biệt cảnh báo liên tiếp và không liên tiếp) chứ không chỉ chép lại. Câu thứ ba trả lời “không tìm thấy” là *đúng*: điểm rèn luyện được quy định trong một văn bản riêng của công tác sinh viên, không nằm trong Quy chế đào tạo, và prompt đã giữ được mô hình không bịa. Chương 5 phân tích đầy đủ các câu trả lời sai.

4.6 Tổng kết

Tại thời điểm viết báo cáo, cả sáu giai đoạn của pipeline đã chạy được trên Quy chế đào tạo SGU 2021: OCR 17 trang (34 276 ký tự), nổi trang và bỏ phần Quyết định ban hành,

cắt thành 21 Node đúng 21 Điều (0% không rõ Điều), lập chỉ mục 21 vector lưu xuống đĩa, truy hồi top-10, xếp hạng lại còn top-3 (*profile server*), và sinh câu trả lời tiếng Việt có trích dẫn số Điều. Hai vấn đề nêu ở bản báo cáo trước — Điều bị chia đôi giữa hai trang và nhãn Điều 1–3 bị trùng — đã được xử lý trong `load_documents()`. Việc còn lại chủ yếu nằm ở dữ liệu (mới có một văn bản) và ở chất lượng câu trả lời, được đánh giá trong Chương 5.

CHƯƠNG 5. ĐÁNH GIÁ

Chương này đo chất lượng hệ thống theo hai tầng như đã nêu ở Chương 2: (1) *truy hỏi* — đo tự động bằng bộ câu hỏi có nhãn; (2) *câu trả lời* — phân tích thủ công nhật ký hỏi đáp, đối chiếu từng câu trả lời với văn bản quy chế. Toàn bộ số liệu dưới đây là kết quả chạy thật trên máy phát triển (RTX A2000 Laptop 4 GB).

5.1 Đánh giá truy hỏi

5.1.1 Phương pháp

Script `eval/evaluate.py` nạp chỉ mục, lấy retriever của profile đang chọn, và với mỗi câu hỏi kiểm tra Điều đúng có nằm trong k Node đầu tiên hay không:

Đoạn mã 5.1: Hàm tính $\text{Recall}@k$ trong `eval/evaluate.py`.

```
1 K_VALUES = (1, 3, 5, 10)
2
3 def evaluate(questions: list[dict], retriever) -> dict[int, float]:
4     hits = {k: 0 for k in K_VALUES}
5     for q in questions:
6         retrieved = [n.node.metadata.get("dieu") for n in
7                     retriever.retrieve(q["question"])]
8         for k in K_VALUES:
9             if q["dieu_dung"] in retrieved[:k]:
10                 hits[k] += 1
11         if q["dieu_dung"] not in retrieved:
12             print(f"MISS: {q['question']} | đúng:
13                 Điều {q['dieu_dung']} | truy hỏi: {retrieved}")
14     return {k: hits[k] / len(questions) for k in K_VALUES}
```

Hàm so khớp bằng metadata `dieu` chứ không bằng nội dung văn bản, nên chỉ đúng khi mỗi số Điều chỉ xuất hiện một lần trong chỉ mục — điều kiện đã được bảo đảm ở Mục 4.3 (21 Node, mỗi Điều một lần). `evaluate.py` chỉ đo retriever và chỉ đọc bộ câu hỏi thứ nhất. Để có thêm MRR, chạy bộ câu hỏi thứ hai và đo tác động của reranker, nhóm dùng thêm một script đo tạm (không lưu trong repository) gọi lại đúng các hàm `build_or_load_index`, `get_retriever` và `get_postprocessors` của dự án, với cùng định nghĩa $\text{Recall}@k$.

Ba cấu hình được đo:

- **local:** vietnamese-bi-encoder, top-10, không rerank. Chỉ mục được lập riêng cho phép đo vì chỉ mục đang lưu là 1024 chiều.
- **server:** Vietnamese_Embedding, top-10, không rerank. Đây cũng là cấu hình mà `eval/evaluate.py` đo khi `RAG_PROFILE=server`.
- **server + rerank:** như trên, sau đó Vietnamese_Reranker chấm lại 10 Node và giữ 3. Do card 4 GB không chứa đồng thời hai mô hình ở fp32 (Mục 3.5), embedding chạy trên CPU và reranker trên GPU trong phép đo này.

5.1.2 Kết quả

Bộ câu hỏi	Cấu hình	R@1	R@3	R@5	R@10	MRR
Bộ 01 (31 câu)	local	64,5%	87,1%	90,3%	100%	0,770
	server	87,1%	100%	100%	100%	0,935
	server + rerank	87,1%	96,8%	—	—	0,919
Bộ 02 (10 câu)	local	20,0%	50,0%	70,0%	90,0%	0,433
	server	70,0%	90,0%	90,0%	90,0%	0,800
	server + rerank	70,0%	90,0%	—	—	0,800

Bảng 5.1: Kết quả truy hồi. Với rerank, chỉ 3 Node được giữ lại nên R@5 và R@10 không có nghĩa.

Thời gian truy hồi trung bình mỗi câu (đã nạp sẵn mô hình, trên GPU): khoảng 8 ms với profile `local` và 17–50 ms với profile `server`. Cấu hình có rerank mất khoảng 1,2 giây mỗi câu, nhưng con số này gồm cả việc embedding câu hỏi trên CPU nên không dùng để kết luận về chi phí riêng của reranker.

5.1.3 Phân tích

Mô hình embedding quyết định chất lượng truy hồi. Trên bộ 01, chỉ đổi mô hình embedding đã nâng Recall@1 từ 64,5% lên 87,1% và MRR từ 0,770 lên 0,935. Với profile `server`, Điều đúng luôn nằm trong top 3 cho cả 31 câu. Với profile `local`, phải tới $k = 10$ (tức gần một nửa trong 21 Điều) mới tìm thấy hết; bốn câu trượt khỏi top 3, ví dụ “Một học phần được xếp lịch tối đa bao nhiêu giờ mỗi tuần?” (đúng: Điều 6, nhận được Điều 4, 7, 3). Vì $k = 10$ nên LLM vẫn thấy Điều đúng trong ngữ cảnh ở cả hai profile, nhưng với profile `local` nó lẫn giữa nhiều Điều không liên quan hơn.

Reranker không cải thiện trên dữ liệu hiện tại. Trên bộ 01, reranker giữ nguyên Recall@1 và *làm tụt* một câu ra khỏi top 3: “Đào tạo vừa làm vừa học kéo dài hơn chính quy bao nhiêu phần trăm?” (đúng: Điều 2) bị xếp sau Điều 4, 20, 5 — Điều 4 có tên “Hình thức đào tạo” và nói nhiều về vừa làm vừa học, nên cross-encoder đánh giá cao hơn. Trên bộ 02, kết quả không đổi. Lý do hợp lý là kho chỉ có 21 Node, retriever đã đặt Điều đúng vào top 3 ở hầu hết câu hỏi, nên reranker gần như không còn chỗ để cải thiện mà vẫn có thể làm hỏng. Reranker chỉ đáng giá khi kho lớn hơn nhiều (nhiều văn bản, Node nhỏ hơn); với dữ liệu hiện tại nó tốn thêm một mô hình trên GPU và loại bỏ 7 trong 10 Node mà không đem lại lợi ích đo được.

Số liệu của bộ 02 cao hơn thực tế. Bộ 02 hỏi về nội dung *sửa đổi* bởi Quyết định 3183/2025: thời hạn rút học phần theo quy định mới, thang điểm chữ bổ sung B+, C+, D+... Quyết định này *chưa có trong chỉ mục*; văn bản Quy chế 2021 không hề chứa “B+”, “C+” hay “D+”. Nhân *dieu_dung* của bộ 02 là số Điều của Quy chế 2021 bị *sửa đổi* (ví dụ Điều 9 về thang điểm), nên retriever “trúng” Điều 9 của bản cũ được tính là đúng dù nội dung Node đó không chứa đáp án. Câu cuối của bộ 02 có nhãn “2 (QĐ 3183)” không thể khớp, nên Recall tối đa là 90%. Như vậy bộ 02 hiện chỉ cho biết retriever tìm đúng *Điều cần sửa đổi*; muốn trả lời được các câu này phải nạp Quyết định 3183/2025 vào chỉ mục (Mục 5.4).

5.2 Đánh giá câu trả lời

5.2.1 Dữ liệu

Nhập ký `eval/results/test-basic.md` ghi lại một phiên chạy `src/pipeline.py` ở profile server (có reranker, nên mỗi câu trả lời dựa trên 3 Điều): 56 câu hỏi do nhóm tự đặt theo cách sinh viên thật sự hỏi — nhiều câu viết tắt (“bao nhiêu chỉ”), sai chính tả (“xuất sất”, “dặt điều khiến trúng tiền”), câu hỏi nối tiếp (“còn kỹ sư”), và cả câu ngoài phạm vi (học phí, điểm rèn luyện) hoặc không phải câu hỏi (“chán”). Bộ câu hỏi này không có nhãn nên được chấm thủ công bằng cách đối chiếu với bản OCR của Quy chế.

Trong 56 câu, hệ thống trả lời “Không tìm thấy trong quy chế” ở 29 câu (52%) và đưa ra câu trả lời ở 27 câu.

5.2.2 Các trường hợp tiêu biểu

Câu hỏi	Hệ thống trả lời	Quy chế 2021 quy định	Đánh giá
Học cử nhân cần bao nhiêu tín chỉ	Tối thiểu 120 tín chỉ	120 tín chỉ (Điều 2)	Đúng
Muốn được B cần bao nhiêu điểm	Từ 7,0 đến 8,4	B: 7,0–8,4 (Điều 9)	Đúng
Cảnh báo kỳ 1 và 3, kỳ 2 bình thường có bị buộc thôi học	Không, vì hai lần không liên tiếp	Buộc thôi học khi quá hai lần cảnh báo <i>liên tiếp</i> (Điều 11)	Đúng
Mua bằng tiếng Anh giả nộp cho trường	Không được; buộc thôi học, thu hồi bằng	Như câu trả lời (Điều 19)	Đúng
8,4 hệ 10 thì hệ 4 được bao nhiêu	3,4 điểm	8,4 là điểm B, B quy đổi thành 3 (Điều 9, 10)	Sai
Khối lượng học kỳ chính	Tối thiểu 30 tuần	2 học kỳ chính <i>cộng lại</i> tối thiểu 30 tuần (Điều 6)	Sai
Học bị F thì làm gì	Học lại; điểm lần học cuối là điểm chính thức	Điểm chính thức là điểm <i>cao nhất</i> giữa các lần học (Điều 9)	Sai một phần
Học điểm P thì sao	P không tính vào điểm TB, “quy đổi thành 0”	P không được tính vào điểm TB (Điều 10 khoản 3), không quy đổi	Sai một phần
Ra trường xuất sắc cần bao nhiêu điểm	Không tìm thấy	Từ 3,6 đến 4,0 (Điều 10), và Điều 10 có trong 3 Điều nguồn	Từ chối sai
Còn kỹ sư (hỏi tiếp câu cử nhân)	Không tìm thấy	150 tín chỉ; hỏi lại đầy đủ thì trả lời đúng	Từ chối sai
Điểm rèn luyện là gì	Không tìm thấy	Không thuộc Quy chế đào tạo (văn bản riêng của CTSV)	Từ chối đúng
Bao nhiêu tiền một tín chỉ	Không tìm thấy	Không thuộc Quy chế đào tạo	Từ chối đúng

Bảng 5.2: Một số câu trả lời trong nhật ký, đối chiếu với văn bản Quy chế 2021.

5.2.3 Phân tích

Prompt chống bịa hoạt động, nhưng hơi quá tay. Với câu hỏi ngoài phạm vi (điểm rèn luyện, học phí, “nên học lộ trình nào”), hệ thống từ chối thay vì bịa — đúng mục tiêu đặt ra trong prompt. Nhưng tỷ lệ từ chối 52% cao hơn tỷ lệ câu thật sự ngoài phạm vi: có những câu mà thông tin *đã nằm trong ngữ cảnh* vẫn bị từ chối, như câu hỏi về điểm ra trường loại xuất sắc. Câu này viết sai chính tả (“xuất sắ”), cho thấy mô hình 8B dễ từ chối khi câu hỏi không khớp chữ với văn bản.

Lỗi sai tập trung ở phép tính và chi tiết số. Các câu trả lời sai đều có Điều đúng trong ngữ cảnh (truy hồi đúng), nhưng LLM đọc sai hoặc suy diễn thêm: tự “quy đổi” 8,4 thành 3,4 thay vì tra bảng quy đổi, đọc “30 tuần cho 2 học kỳ” thành “30 tuần mỗi học kỳ”, hay nhớ quy tắc “lấy điểm lần cuối” (phổ biến ở nơi khác) thay cho “lấy điểm cao nhất” của SGU. Đây là loại lỗi nguy hiểm nhất vì câu trả lời vẫn trích dẫn đúng Điều nên trông rất đáng tin. Người dùng cần được nhắc đối chiếu Điều được trích, và nên thử prompt yêu cầu trích nguyên văn câu quy định trước khi kết luận.

Không có ngữ cảnh hội thoại. `pipeline.py` gửi từng câu hỏi độc lập cho `RetrieverQueryEngine`, nên câu nối tiếp “còn kỹ sư” (sau câu hỏi về cử nhân) không có đủ thông tin để truy hồi và bị từ chối; hỏi lại đầy đủ “tôi học kỹ sư thì cần học bao nhiêu chỉ” thì trả lời đúng 150 tín chỉ. Sinh viên thường hỏi nối tiếp như vậy, nên đây là hạn chế đáng kể.

Câu trả lời không ổn định. Cùng một câu hỏi “tôi học rớt một môn nhưng các kỳ còn lại 4.0 thì có được xuất sắc không” được hỏi hai lần, chỉ khác nhau một dấu hỏi chấm và khoảng trắng: lần đầu hệ thống từ chối, lần sau trả lời “Không”. Câu trả lời “Không” này cũng chưa đúng với Quy chế: hạng tốt nghiệp xuất sắc chỉ bị hạ một mức khi khối lượng học lại vượt 5% tổng tín chỉ hoặc người học bị kỷ luật (Điều 13), và điểm học lại được lấy theo lần cao nhất.

5.3 Hạn chế

- **Dữ liệu:** chỉ mục mới có một văn bản (Quy chế 2021). Các câu hỏi về quy định mới (Quyết định 3183/2025) và về công tác sinh viên (điểm rèn luyện, học bổng, học phí) chưa trả lời được.

- **Đánh giá:** bộ câu hỏi có nhãn nhỏ (41 câu) và chỉ chấm truy hồi; câu trả lời mới được chấm thủ công trên một phiên. Bộ 02 cho số liệu cao hơn thực tế như đã phân tích.
- **Hỏi đáp:** không nhớ ngữ cảnh hội thoại; LLM 8B sai ở các phép quy đổi số; câu trả lời chưa ổn định giữa các lần hỏi.
- **Hạ tầng:** hai profile dùng chung thư mục chỉ mục dù khác số chiều vector; profile `server` ở fp32 không vừa card 4 GB khi bật reranker; LLM phụ thuộc máy GPU thuê.
- **Giao diện:** mới có dòng lệnh, chưa có giao diện web.

5.4 Hướng phát triển

Một số việc đã được làm thử trên các nhánh phát triển khác của repository và chưa gộp vào nhánh chính:

- Nạp embedding và reranker ở fp16 để cả hai cùng nằm trên card 4 GB (khoảng 2,2 GB).
- Tách embedding và reranker thành một server HTTP (FastAPI) để nạp mô hình một lần, các tiến trình hỏi đáp và đánh giá chỉ gọi qua HTTP. Khi thử, vector do server trả về trùng với vector tính tại chỗ ($\text{cosine} \geq 0,99999$).
- Tổ chức lại 305 tệp tài liệu đã thu thập từ website đào tạo, công tác sinh viên và các khoa thành năm nhóm (quy chế đào tạo, công tác sinh viên, chương trình đào tạo, khóa luận–thực tập, biểu mẫu), kèm file metadata ghi URL gốc và các bản trùng nội dung.

Các bước tiếp theo theo thứ tự ưu tiên:

1. Nạp Quyết định 3183/2025 và gắn metadata tên văn bản, ngày hiệu lực vào Node; khi một Điều đã bị sửa đổi, ưu tiên nội dung mới. Sau đó sửa nhãn của bộ 02 để chấm theo văn bản, không chỉ theo số Điều.
2. Thay `RetrieverQueryEngine` bằng chat engine có viết lại câu hỏi theo lịch sử hội thoại (ví dụ chế độ *condense question* của LlamaIndex) để xử lý câu hỏi nối tiếp.

3. Mở rộng sang các văn bản công tác sinh viên, kèm bộ định tuyến (router) chọn chỉ mục theo loại câu hỏi.
4. Xây bộ đánh giá câu trả lời có đáp án chuẩn, chấm tự động bằng LLM hoặc theo từ khóa số liệu.
5. Tách thư mục chỉ mục theo profile; gỡ trường `corpus_sources` không dùng trong `config.py`.
6. Làm giao diện Gradio hiển thị câu trả lời kèm nguyên văn các Điều được trích.

5.5 Kết luận

Đồ án đã xây dựng được một hệ RAG hoàn chỉnh trên LlamaIndex cho Quy chế đào tạo SGU 2021, chạy bằng mô hình mở: OCR văn bản scan, cắt theo Điều, lập chỉ mục vector, truy hồi, sinh câu trả lời tiếng Việt có trích dẫn số Điều. Ở bước truy hồi, với mô hình AI-TeamVN/Vietnamese_Embedding, Điều đúng nằm trong top 3 ở cả 31 câu của bộ 01 (Recall@1 87,1%, MRR 0,935). Điểm yếu hiện nay không còn ở truy hồi mà ở dữ liệu (mới có một văn bản) và ở bước sinh: LLM 8B đôi khi đọc sai số liệu hoặc từ chối dù ngữ cảnh có đáp án. Đây là hai hướng cần tập trung tiếp theo.